100 MOST IMPORTANT SPRING & SPRING BOOT QUESTIONS (INTERVIEW-READY ANSWERS)

SPRING CORE (1–20)

1. What is Spring Framework?

Spring is a lightweight Java framework used for building enterprise applications. It follows Dependency Injection and IoC principles to reduce tight coupling. Spring handles infrastructure concerns like transactions, security, and bean lifecycle. This allows developers to focus mainly on business logic.

2. What is Dependency Injection?

Dependency Injection means dependencies are provided by the container instead of being created manually. This reduces coupling between classes and improves testability. In Spring, DI is managed by the IoC container. It supports constructor, setter, and field injection.

3. Why is Dependency Injection important?

DI makes applications loosely coupled and easier to maintain. It improves unit testing by allowing dependencies to be mocked. It also follows SOLID principles, especially Dependency Inversion. Large applications become more scalable using DI.

4. What is Inversion of Control?

IoC means control of object creation and lifecycle is moved from the application code to the Spring container. Developers do not manage object creation manually. Spring decides when and how objects are created. This improves flexibility and maintainability.

5. What is a Spring Bean?

A Spring Bean is a Java object managed by the Spring IoC container. Spring controls its lifecycle, dependency injection, and destruction. Beans are defined using annotations or configuration classes. Centralized management improves consistency.

6. What is ApplicationContext?

ApplicationContext is the main Spring container interface. It manages beans, configuration, events, and resources. It eagerly initializes singleton beans by default. It also supports AOP and internationalization.

7. Difference between BeanFactory and ApplicationContext?

BeanFactory provides basic DI functionality. ApplicationContext extends it with advanced features like event publishing, AOP, and message sources. ApplicationContext is preferred for enterprise applications. BeanFactory is rarely used directly.

8. What are bean scopes?

Bean scope defines the lifecycle of a bean. Singleton creates one instance per container. Prototype creates a new instance each time. Web scopes include request, session, and application.

9. What is bean lifecycle?

Bean lifecycle includes instantiation, dependency injection, initialization, usage, and destruction. Spring provides hooks like @PostConstruct and @PreDestroy. The container manages these phases automatically.

10. Constructor vs Field Injection?

Constructor injection ensures required dependencies are available at object creation. It promotes immutability and easier testing. Field injection is simpler but harder to test. Constructor injection is recommended.

11. What is @Component?

@Component marks a class as a Spring-managed bean. It allows Spring to detect the class during component scanning. It is a generic stereotype annotation. Specialized annotations are built on top of it.

12. Difference between @Component, @Service, @Repository?

@Service is used for business logic. @Repository is used for persistence layer and enables exception translation. @Component is generic. Functionally similar, but semantically different.

13. What is @Autowired?

@Autowired automatically injects dependencies by type. Spring resolves the dependency from the IoC container. It can be used on constructors, fields, or setters. Constructor usage is preferred.

14. What is @Qualifier?

@Qualifier is used when multiple beans of the same type exist. It helps Spring identify which bean to inject. It is used along with @Autowired. This avoids ambiguity errors.

15. What is @Primary?

@Primary marks a bean as the default choice. When multiple beans exist, Spring injects the primary one. It reduces the need for @Qualifier. Useful for default implementations.

16. What is @Lazy?

@Lazy delays bean initialization until it is required. By default, singleton beans are eagerly loaded. Lazy loading improves startup time. Useful for heavy beans.

17. What is AOP?

Aspect-Oriented Programming separates cross-cutting concerns from business logic. Examples include logging, security, and transactions. Spring uses proxies to implement AOP. This improves code cleanliness.

18. What are cross-cutting concerns?

Cross-cutting concerns affect multiple parts of the application. Logging, security, and transaction management are common examples. AOP handles these separately. This avoids code duplication.

19. What is @Configuration?

@Configuration indicates a class that defines bean configurations. It is processed by the Spring container. It ensures @Bean methods return singleton instances. Used in Java-based configuration.

20. What is @Bean?

@Bean explicitly defines a bean inside a configuration class. It gives more control over bean creation. Used when third-party classes need to be registered. It complements component scanning.

SPRING BOOT (21–45)

21. What is Spring Boot?

Spring Boot is built on top of Spring to simplify application development. It provides auto-configuration and embedded servers. It reduces boilerplate code and setup time. Widely used for microservices.

22. Why use Spring Boot?

Spring Boot speeds up development by reducing configuration. It provides production-ready features. Developers can run applications as standalone services. It improves developer productivity.

23. Difference between Spring and Spring Boot?

Spring requires manual configuration. Spring Boot provides auto-configuration and defaults. Boot eliminates XML configuration. Boot applications are easier to deploy.

24. What is auto-configuration?

Auto-configuration automatically configures beans based on classpath dependencies. Spring Boot checks available libraries. It applies sensible defaults. Developers can override if needed.

25. What is @SpringBootApplication?

It combines @Configuration, @EnableAutoConfiguration, and @ComponentScan. It marks the main application class. It triggers auto-configuration and scanning. Simplifies bootstrap.

26. What is embedded server?

Spring Boot provides embedded servers like Tomcat. Applications run as executable JARs. No need for external server deployment. Simplifies DevOps.

27. What is application.properties?

It externalizes configuration values. Used for database, server, and logging configs. Can vary per environment. Supports overriding without code changes.

28. application.properties vs application.yml?

YAML is hierarchical and more readable. Properties use simple key-value pairs. Both serve the same purpose. Choice depends on preference.

29. What are Spring Boot starters?

Starters are dependency descriptors. They bundle commonly used dependencies. They reduce version conflicts. Example: spring-boot-starter-web.

30. What is Spring Boot Actuator?

Actuator provides monitoring and management endpoints. It exposes health, metrics, and environment data. Used in production systems. Helps with observability.

31. Common Actuator endpoints?

/health, /metrics, /info, /env are common endpoints. They provide runtime insights. Used by monitoring tools. Can be secured.

32. What is Spring Boot DevTools?

DevTools improves developer productivity. It supports auto-restart and live reload. Used only in development. Not recommended for production.

33. What is CommandLineRunner?

It runs code after application startup. Used for initialization logic. Common for loading test data. Executes once.

34. What is fat JAR?

Fat JAR contains application code and dependencies. Allows standalone execution. Simplifies deployment. Common in Spring Boot.

35. JAR vs WAR?

JAR uses embedded server. WAR is deployed to external server. JAR is preferred in microservices. WAR is used in legacy setups.

36. How does Spring Boot handle logging?

Spring Boot uses Logback by default. Logging levels are configurable. Logs can be externalized. Easy integration with monitoring tools.

37. What is profile?

Profiles allow environment-specific configuration. Examples: dev, test, prod. Helps isolate configurations. Activated via properties.

38. How to activate profiles?

Profiles are activated using spring.profiles.active. Can be set via properties or environment variables. Supports multiple profiles.

39. What is externalized configuration?

Configuration is moved outside code. Enables environment flexibility. Supports properties, YAML, env variables. Essential for cloud apps.

40. What is Spring Boot testing support?

Spring Boot provides testing annotations. It simplifies integration testing. Supports mocking and context loading. Improves test coverage.

41. What is @SpringBootTest?

It loads the full application context. Used for integration tests. Slower but comprehensive. Tests real behavior.

42. What is @WebMvcTest?

It tests only controller layer. Loads minimal context. Faster than full context tests. Used for API testing.

43. How do you secure Actuator endpoints?

Using Spring Security. Restrict access by role. Disable unnecessary endpoints. Important for production safety.

44. What is HikariCP?

HikariCP is a high-performance JDBC connection pool. It is default in Spring Boot. Improves database performance. Lightweight and fast.

45. How do you improve Spring Boot startup?

Use lazy initialization. Reduce auto-configurations. Use lighter dependencies. Optimize component scanning.

REST, JPA, SECURITY (46–70)

46. What is Spring MVC?

Spring MVC is a web framework based on MVC pattern. It handles HTTP requests. Separates concerns. Used for REST APIs.

47. What is @RestController?

@RestController creates REST APIs. It combines @Controller and @ResponseBody. Automatically returns JSON. Simplifies API development.

48. Difference between @Controller and @RestController?

@Controller returns views. @RestController returns data. REST is used for APIs. MVC is used for UI.

49. What is @RequestMapping?

Maps HTTP requests to controller methods. Supports multiple HTTP methods. Base annotation for mappings. Flexible routing.

50. What is @GetMapping?

Shortcut for GET requests. Improves readability. Used in REST APIs. Reduces boilerplate.

51. What is @RequestBody?

Maps request body to Java object. Used in POST and PUT. Automatically deserializes JSON. Simplifies input handling.

52. What is @PathVariable?

Extracts values from URL path. Used for dynamic routes. Common in REST APIs. Improves URL clarity.

53. What is @RequestParam?

Extracts query parameters. Used for filtering and pagination. Optional or required. Simple request handling.

54. What is ResponseEntity?

Represents full HTTP response. Includes status, headers, and body. Provides more control. Common in REST APIs.

55. How do you handle exceptions globally?

Using @ControllerAdvice. Centralizes exception handling. Ensures consistent error responses. Improves code cleanliness.

56. What is Spring Data JPA?

Simplifies database access. Reduces boilerplate code. Uses repository abstraction. Built on JPA.

57. What is JpaRepository?

Provides CRUD operations. Supports pagination and sorting. Reduces manual queries. Extends repository interfaces.

58. Difference between JPA and Hibernate?

JPA is a specification. Hibernate is an implementation. JPA defines rules. Hibernate provides features.

59. What is @Entity?

Marks a class as a database entity. Maps to a table. Used by JPA. Essential for ORM.

60. Lazy vs Eager loading?

Lazy loads data when needed. Eager loads immediately. Lazy improves performance. Eager can cause overhead.

61. What is N+1 problem?

Occurs when lazy loading triggers multiple queries. Impacts performance. Common in ORM. Needs optimization.

62. How to solve N+1 problem?

Use fetch joins. Use EntityGraph. Optimize queries. Reduce unnecessary database calls.

63. What is @Transactional?

Manages transactions declaratively. Ensures atomic operations. Handles rollback automatically. Maintains consistency.

64. What is transaction propagation?

Defines behavior of nested transactions. Example: REQUIRED, REQUIRES_NEW. Controls transaction flow. Important in services.

65. What is transaction isolation?

Controls data visibility. Prevents dirty reads. Defines concurrency behavior. Important for data integrity.

66. What is Spring Security?

Framework for authentication and authorization. Secures applications. Integrates with Spring Boot. Highly customizable.

67. Authentication vs Authorization?

Authentication verifies identity. Authorization checks permissions. Both are security concepts. Used together.

68. What is JWT?

JWT is a stateless authentication token. Used in REST APIs. Stored on client side. Improves scalability.

69. Why JWT for REST APIs?

REST APIs are stateless. JWT avoids server sessions. Improves performance. Easy to scale.

70. What is CSRF?

CSRF is a security attack. Forces authenticated users to perform actions. Relevant in session-based apps. Usually disabled for REST.

Spring Core / Internals

71. Difference between @Bean and @Component?

@Component is used for automatic bean detection through component scanning.
@Bean is used inside @Configuration classes to explicitly define beans.
@Bean gives more control over bean creation logic.
It is commonly used for third-party or complex object creation.

72. How does Spring resolve @Autowired internally?

Spring first checks the application context for matching bean types.
If multiple beans are found, it looks for @Primary or @Qualifier.
If ambiguity remains, it throws an exception.
Resolution happens during context initialization.

73. What happens if two beans have the same name?

Spring will throw a BeanDefinitionOverrideException.
By default, bean overriding is disabled in Spring Boot.
It can be enabled using configuration, but it is not recommended.
Unique bean names are considered best practice.

74. How does Spring handle circular dependencies?

Spring can resolve circular dependencies with setter or field injection.
It cannot resolve them with constructor injection.
Spring uses early bean references to solve this.
Constructor-based circular dependencies cause startup failure.

75. What is BeanPostProcessor?

BeanPostProcessor allows custom logic before and after bean initialization.
It is used for modifying beans at runtime.
Spring internally uses it for features like @Autowired.
Commonly used in frameworks and libraries.

76. What is FactoryBean?

FactoryBean is used when bean creation logic is complex.
It returns an object instead of itself.
Spring calls getObject() to create the bean.
Used internally for proxies and special objects.

77. Difference between eager and lazy initialization?

Eager initialization creates beans at startup.
Lazy initialization creates beans only when required.
Lazy loading improves startup performance.
Configured using @Lazy annotation.

78. How does Spring handle null dependencies?

Spring throws an exception if a required dependency is missing.
Optional dependencies can be handled using Optional or @Autowired(required=false).
Null safety must be handled carefully.
Constructor injection prevents null issues.

79. What is the purpose of @DependsOn?

@DependsOn ensures one bean is initialized before another.
Used when bean initialization order matters.
It controls startup sequence.
Useful in legacy or integration scenarios.

80. Difference between @Value and Environment?

@Value injects individual property values.
Environment provides programmatic access to properties.
Environment is more flexible.
Used when dynamic property access is required.

Spring Boot & Configuration

81. How does Spring Boot decide auto-configuration?

Spring Boot checks the classpath for dependencies.
It uses conditional annotations like @ConditionalOnClass.
Configurations are applied only if conditions match.
This reduces unnecessary bean creation.

82. What is spring.factories?

spring.factories lists auto-configuration classes.
Spring Boot loads them at startup.
It enables modular auto-configuration.
Used internally by starters.

83. How do you exclude auto-configuration?

Auto-configuration can be excluded using exclude attribute.
Can also be disabled using properties.
Useful when defaults are not needed.
Prevents unnecessary bean creation.

84. How does Spring Boot load application.properties?

Spring Boot loads properties during startup.
It follows a defined priority order.
Properties are stored in the Environment.
Accessible throughout the application.

85. Property resolution order in Spring Boot?

Command-line arguments have highest priority.
Then environment variables.
Then application properties.
Default values have the lowest priority.

86. How do you override properties at runtime?

Using command-line arguments or environment variables.
Profiles can also override values.
External config files can be used.
Common in cloud deployments.

87. Purpose of @ConfigurationProperties?

It binds external configuration to POJO classes.
Supports type safety and validation.
Better than using multiple @Value.
Used for complex configuration.

88. @ConfigurationProperties vs @Value?

@Value is for single values.
@ConfigurationProperties handles groups of properties.
The latter is cleaner and scalable.
Recommended for large configs.

89. What is a custom starter?

A custom starter bundles dependencies and auto-configuration.
Used to share common setup across projects.
Follows Spring Boot starter conventions.
Improves reusability.

90. What happens during Spring Boot startup?

Spring Boot initializes the context.
Auto-configuration is applied.
Beans are created and injected.
Application is ready to serve requests.

REST, Performance & Production

91. How do you version REST APIs?

Versioning can be done using URI or headers.
URI versioning is most common.
Helps maintain backward compatibility.
Essential for long-term APIs.

92. Pagination and sorting in REST APIs?

Implemented using Spring Data pagination.
Pageable and Sort are used.
Improves performance for large datasets.
Common in list APIs.

93. How do you handle validation?

Using @Valid and validation annotations.
Errors are handled globally.
Ensures request data integrity.
Common in REST APIs.

94. How do you handle large file uploads?

Use multipart configuration.
Set max file size limits.
Stream files instead of loading into memory.
Important for performance.

95. How do you improve REST API performance?

Use caching and pagination.
Optimize database queries.
Use async processing where possible.
Monitor using Actuator.

96. How do you enable async processing?

Enable using @EnableAsync.
Use @Async on methods.
Runs methods in separate threads.
Improves responsiveness.

97. How does @Async work internally?

Spring creates a proxy around the bean.
Method executes in a separate thread.
Uses task executor.
Caller does not wait for response.

98. How do you handle timeouts?

Configure connection and read timeouts.
Use circuit breakers.
Handle fallback responses.
Important for resilience.

99. How do you log request and response safely?

Use filters or interceptors.
Mask sensitive data.
Avoid logging payloads fully.
Follow security best practices.

100. How do you troubleshoot production issues?

Use logs and metrics.
Check Actuator endpoints.
Analyze thread dumps.
Monitor system health continuously.